

=CO-4

Java Database Connectivity (JDBC), Servlets and JSP

PostgreSQL Link: https://www.w3schools.com/postgresql/postgresql_create_table.php

Main =Topics:

1. Limitations of File System and the need for JDBC API in application development –
2. JDBC - Architecture (2 Tier & 3 Tier) and Drivers & Its Types -----
3. Steps to connect to the PostgreSQL database and perform CRUD Operations-----
4. An Overview of Client-Server Communication and Servlet-----
5. Servlet demo Application development with JSP Form Data-----
6. Servlet demo Application development with JDBC to persist data to the database-
7. JSP & Advantages over servlets, features, syntax, Lifecycle of JSP-----
8. JSP tags-----
9. Interaction between JDBC, Servlet, and JSP Form Data for CRUD Operation-----



K. J. SOMAIYA INSTITUTE OF TECHNOLOGY AND ENGINEERING

Limitations of File System and the need for JDBC API in application development

Limitations of File System in Application Development:

- When building applications, one common approach for storing and retrieving data is using the file system. However, using file systems for data storage has several limitations, which can hinder scalability, performance, and reliability. These limitations include:

a). Data Organization and Structure:

- Files in the file system are unstructured or semi-structured, meaning you have to manually manage how the data is organized and retrieved.
- Unlike databases, files do not support advanced querying or indexing, making complex operations (like searching or filtering) more difficult and slower.

Example: Searching for all records that meet certain conditions would require opening each file and parsing the contents, which can be very inefficient.

```
// Example: Searching for a particular entry in a file system
File file = new File("data.txt");
BufferedReader br = new BufferedReader(new FileReader(file));
String line;
while ((line = br.readLine()) != null) {
    if (line.contains("specificCriteria")) {
        System.out.println(line);
    }
}
br.close();
```

b). Concurrency Issues:

- File systems are not built to handle concurrent access efficiently. If multiple threads or users try to access or modify the same file simultaneously, it can lead to data corruption or race conditions unless manual locking mechanisms are implemented.
- Managing file locking manually adds complexity and can lead to performance bottlenecks.

```
// Example of manual file locking
File file = new File("data.txt");
try (RandomAccessFile raf = new RandomAccessFile(file, "rw");
     FileChannel channel = (FileChannel) raf.getChannel();
     FileLock lock = channel.lock()) {
    // Critical section where the file is locked for writing
    raf.writeBytes("New data");
} catch (IOException e) {
    e.printStackTrace();
}
```

c). Security:

- File systems have basic security controls based on operating system-level permissions. These controls are not sufficient for most applications that need fine-grained access control, encryption, or audit trails.
- Controlling access at the file level does not provide flexibility to apply permissions at a more granular level (like specific data records).

d). Data Integrity and Reliability:

- Files can become corrupted, and there are no built-in mechanisms for recovery in case of failure or crash.

- Ensuring data integrity, atomic operations, and transaction support requires complex manual implementation.

```
// Example of handling file corruption manually
try {
    File file = new File("data.txt");
    BufferedReader br = new BufferedReader(new FileReader(file));
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    // Handle corrupted file or I/O errors
    System.err.println("File is corrupted or cannot be read");
}
```

e). Scalability:

- As the size and number of files grow, it becomes challenging to manage them. File systems are not optimized for high-performance access to very large datasets.
- Performing frequent read/write operations can slow down as more files or larger files are added.

f). Lack of Advanced Features:

- File systems do not provide advanced features like transactions, relationships between data (e.g., foreign keys in databases), constraints (e.g., primary keys), and automatic backups.

Need for JDBC API in Application Development

Due to the limitations of file systems, applications often require a more robust and scalable way to store and manage data. This is where JDBC (Java Database Connectivity) comes into play. JDBC is an API that allows Java applications to interact with relational databases (such as MySQL, PostgreSQL, Oracle, etc.). It provides a standardized way to query, update, and manage databases, overcoming the limitations of file-based storage.

Benefits of Using JDBC Over File Systems:

a). Structured Data Storage:

- Relational databases store data in structured formats (tables, rows, and columns), which makes it easier to manage, query, and retrieve data in an organized manner.
- SQL can be used to perform complex queries, filtering, sorting, and aggregation of data without manually parsing files.

```
// Example of querying a database using JDBC
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "user", "password");
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users WHERE age > 30");

while (rs.next()) {
    System.out.println("User ID: " + rs.getInt("id"));
    System.out.println("Name: " + rs.getString("name"));
}
rs.close();
stmt.close();
conn.close();
```

b). Concurrency Control:

- Databases handle concurrent access efficiently using built-in mechanisms such as transactions, isolation levels, and locking.
- This allows multiple users or processes to read and write data simultaneously without conflicts or data corruption.

```
// Example of a transaction in JDBC
Connection conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/mydb", "user", "password");
conn.setAutoCommit(false); // Start transaction

try {
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("UPDATE accounts SET balance = balance - 100 WHERE id = 1");
    stmt.executeUpdate("UPDATE accounts SET balance = balance + 100 WHERE id = 2");
    conn.commit(); // Commit transaction
} catch (SQLException e) {
    conn.rollback(); // Rollback transaction in case of error
} finally {
    conn.close();
}
```

c). Security:

- Databases offer advanced security features, including user authentication, access control, encryption, and auditing.
- Different users can be granted different levels of access to data, ensuring secure and controlled access to sensitive information.

d). Data Integrity and Reliability:

- Databases support transactions, which ensure atomicity, consistency, isolation, and durability (ACID properties). This ensures that data remains consistent even in the case of failures.
- Features like foreign keys, constraints, and indexes ensure data integrity and allow for efficient querying.

```
// Example of enforcing data integrity through a foreign key
String createTableSQL = "CREATE TABLE Orders ("
    + "OrderID int NOT NULL, "
    + "CustomerID int, "
    + "PRIMARY KEY (OrderID), "
    + "FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID))";

Statement stmt = conn.createStatement();
stmt.execute(createTableSQL);
}
```

e). Scalability:

- Databases are designed to handle large amounts of data efficiently. Indexes, partitioning, and clustering allow databases to scale and perform well, even with increasing data volumes.
- File systems struggle with large datasets, while databases provide high-performance solutions to store, retrieve, and manage big data.

f). Advanced Features:

Databases provide many features not available in file systems, such as:

- *Joins* (combining data from multiple tables)
- *Views* (virtual tables based on SQL queries)
- *Stored Procedures* (predefined SQL scripts that can be executed)
- *Triggers* (automatic actions triggered by specific changes in data)

JDBC - Architecture (2 Tier & 3 Tier) and Drivers & Its Types

Introduction to JDBC (Java Database Connectivity)

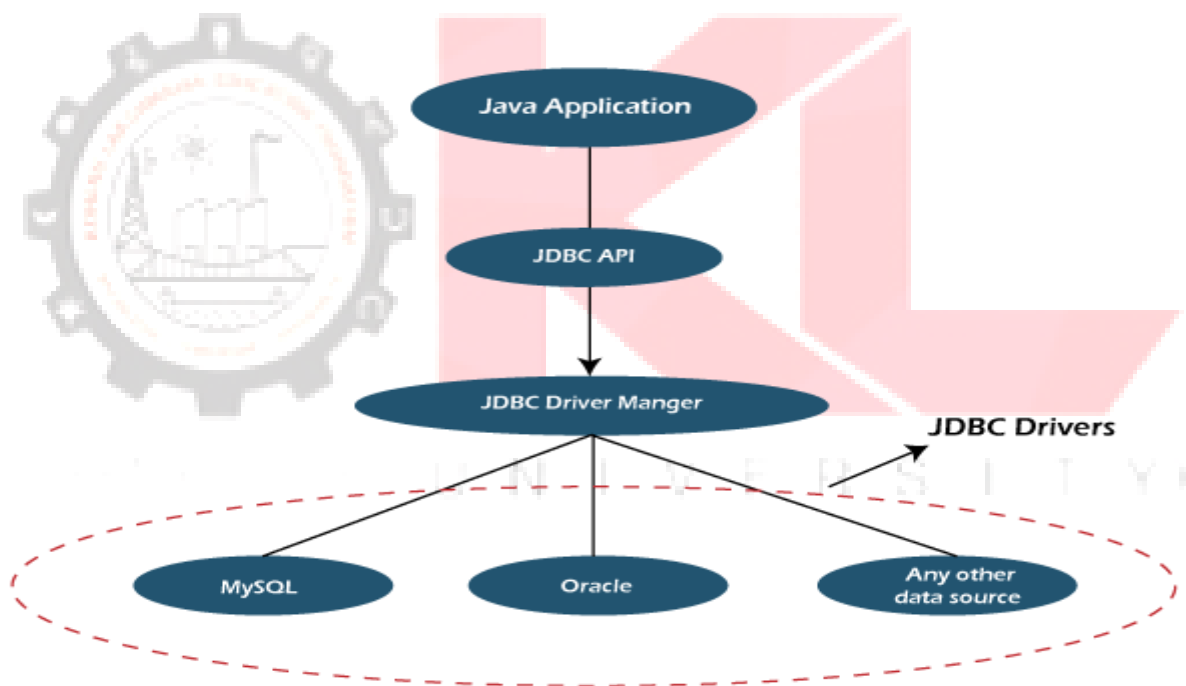
- JDBC stands for Java Database Connectivity.
- JDBC is a Java API to connect and execute the query with the database.
- It is a specification from Sun Microsystems that provides a standard abstraction (API or Protocol) for Java applications to communicate with various databases.

Definition of JDBC (Java Database Connectivity)

- JDBC is an API (Application programming interface) used in Java programming to interact with databases. The classes and interfaces of JDBC allow the application to send requests made by users to the specified database.
- The current version of JDBC is JDBC 4.3, released on 21st September 2017.

JDBC API: It provides various methods and interfaces for easy communication with the database. It provides two packages as follows, which contain the java SE and Java EE platforms to exhibit WORA(write once run anywhere) capabilities. The **java.sql package** and **javax.sql package** contains interfaces and classes of JDBC API.

Architecture of JDBC:



- Java Application:** It is a java applet or a servlet that communicates with a data source.
- The JDBC API:** The JDBC API allows Java programs to execute SQL statements and retrieve results. Some of the important interfaces defined in JDBC API are as follows: Driver interface , ResultSet Interface , RowSet Interface , PreparedStatement interface, Connection inteface, and Classes defined in JDBC API are as follows: DriverManager class, Types class, Blob class, clob class.
- DriverManager:** It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
- JDBC drivers:** To communicate with a data source through JDBC, you need a JDBC driver that intelligently communicates with the respective data source

JDBC has four major components that are used for the interaction with the database.

1. JDBC API
2. JDBC Test Suite

3. JDBC Driver Manger
4. JDBC ODBC Bridge Driver

1) JDBC API:

JDBC API provides various interfaces and methods to establish easy connection with different databases.

```
javax.sql.*;
java.sql.*;
```

2) JDBC Test suite:

JDBC Test suite facilitates the programmer to test the various operations such as deletion, updating, insertion that are being executed by the JDBC Drivers.

3) JDBC Driver manager:

JDBC Driver manager loads the database-specific driver into an application in order to establish the connection with the database. The JDBC Driver manager is also used to make the database-specific call to the database in order to do the processing of a user request.

4) JDBC-ODBC Bridge Drivers:

JDBC-ODBC Bridge Drivers are used to connect the database drivers to the database. The bridge does the translation of the JDBC method calls into the ODBC method call. It makes the usage of the **sun.jdbc.odbc** package that encompasses the native library in order to access the ODBC (Open Database Connectivity) characteristics.

Types of JDBC Architecture (2-tier and 3-tier)

The JDBC architecture consists of *two-tier* and *three-tier* processing models to access a database. They are as described below:

Two-tier model:

- A java application communicates directly to the data source. The JDBC driver enables the communication between the application and the data source. When a user sends a query to the data source, the answers for those queries are sent back to the user in the form of results.
- The data source can be located on a different machine on a network to which a user is connected. This is known as a client/server configuration, where the user's machine acts as a client, and the machine has the data source running acts as the server.

Three-tier model:

- In this, the user's queries are sent to middle-tier services, from which the commands are again sent to the data source. The results are sent back to the middle tier, and from there to the user.
- This type of model is found very useful by management information system directors

Interfaces and Classes of JDBC API:

- **DriverManager class** – used to load a SQL driver to connect to database.
- **Connection interface** – used to make a connection to the database using database connection string and credentials.
- **Statement interface** – used to make a query to the database.
- **PreparedStatement interface** – used for a query with placeholder values.
- **CallableStatement interface** – used to called stored procedure or functions in database.
- **ResultSet interface** – represents the query results obtained from the database.

- **ResultSetMetaData interface** – represents the metadata of the result set.
- **BLOB class** – represents binary data stored in BLOB format in database table.
- **CLOB class** – represents text data like XML stored in database table

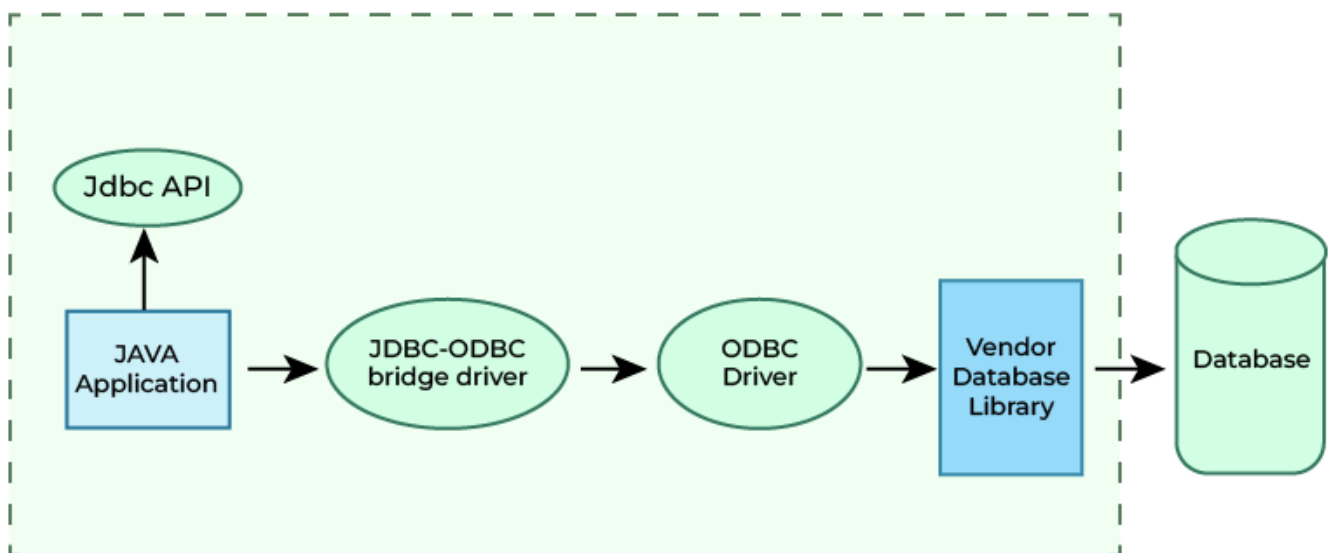
JDBC Drivers

JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand. There are 4 types of JDBC drivers:

1. Type-1 driver or JDBC-ODBC bridge driver
2. Type-2 driver or Native-API driver (partially java driver)
3. Type-3 driver or Network Protocol driver (fully java driver)
4. Type-4 driver or Thin driver (fully java driver)

1. JDBC-ODBC bridge driver – Type 1 driver

Type-1 driver or JDBC-ODBC bridge driver uses ODBC driver to connect to the database. The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. Type-1 driver is also called Universal driver because it can be used to connect to any of the databases.



Advantages

- This driver software is built-in with JDK so no need to install separately.
- It is a database independent driver.

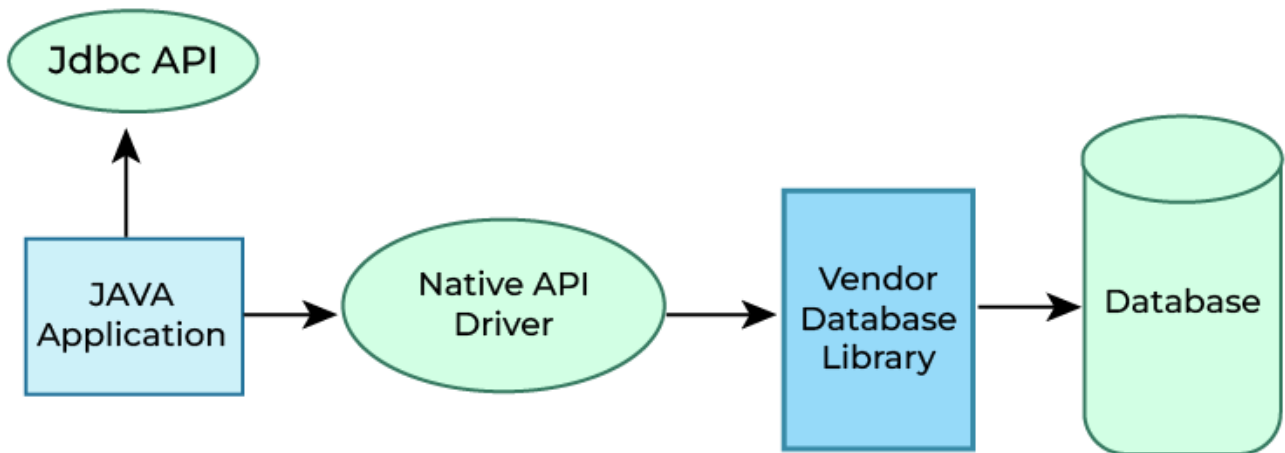
Disadvantages

- As a common driver is used in order to interact with different databases, the data transferred through this driver is not so secured.
- The ODBC bridge driver is needed to be installed in individual client machines.
- Type-1 driver isn't written in java, that's why it isn't a portable driver.

2. Native-API driver – Type 2 driver (Partially Java driver)

The Native API driver uses the client -side libraries of the database. This driver converts JDBC method calls into native calls of the database API. In order to interact with different database, this driver needs their local API, that's why data

transfer is much more secure as compared to type-1 driver. This driver is not fully written in Java that is why it is also called Partially Java driver.



Advantage

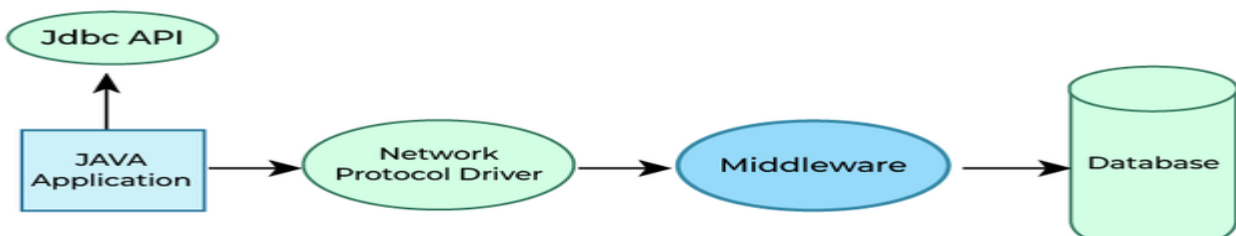
- Native-API driver gives better performance than JDBC-ODBC bridge driver.

Disadvantages

- Driver needs to be installed separately in individual client machines
- The Vendor client library needs to be installed on client machine.
- Type-2 driver isn't written in java, that's why it isn't a portable driver
- It is a database dependent driver.

3. Network Protocol driver – Type 3 driver (fully Java driver)

The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. Here all the database connectivity drivers are present in a single server, hence no need of individual client-side installation.



Advantages

- Type-3 drivers are fully written in Java, hence they are portable drivers.

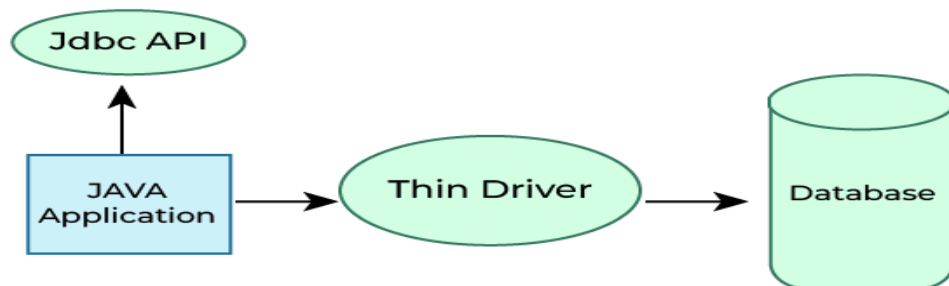
- No client side library is required because of application server that can perform many tasks like auditing, load balancing, logging etc.
- Switch facility to switch over from one database to another database.

Disadvantages

- Network support is required on client machine.
- Maintenance of Network Protocol driver becomes costly because it requires database-specific coding to be done in the middle tier.

4. Thin driver – Type 4 driver (fully Java driver)

Type-4 driver is also called native protocol driver. This driver interact directly with database. It does not require any native database library, that is why it is also known as Thin Driver.



Advantages

- Does not require any native library and Middleware server, so no client-side or server-side installation.
- It is fully written in Java language, hence they are portable drivers.

Disadvantage

- If the database varies, then the driver will carry because it is database dependent.

Which Driver to use When?

- If you are accessing one type of database, such as Oracle, Sybase, or IBM, the preferred driver type is type-4.
- If your Java application is accessing multiple types of databases at the same time, type 3 is the preferred driver.
- Type 2 drivers are useful in situations, where a type 3 or type 4 driver is not available yet for your database.
- The type 1 driver is not considered a deployment-level driver, and is typically used for development and testing purposes only.

Steps to connect to the PostgreSQL database and perform CRUD Operations:

Steps to connect JDBC:

There are 5 steps to connect any java application with the database using JDBC. These steps are as follows:

Step1: Register the Driver class

Step2: Create connection

Step3: Create statement

Step4: Execute queries

Step5: Close connection

Step1:

- Register the Driver: We can register it in 2 ways
 - ✓ `Class.forName("drivername");` → Loads driver into Memory explicitly
 - ✓ `DriverManager.registerDriver("driver object");` → Loads driver into Memory implicitly.
 - ✓ Explicitly Vs Implicitly → What is the difference?

Step2:

- Create a Connection:
- Each database has unique **driver name** & **URL**.
 - ✓ `Connection con = DriverManager.getConnection("URL", "username", "password")`
 - ✓ URL → `"jdbc:postgresql://127.0.0.1/Employee"`
 - ✓ Username → `"postgres"`
 - ✓ Password → `"postgres"`

Step3:

- Create Statement Object for Connection:
 - ✓ `Statement stmt = con.createStatement();`
 - ✓ "`stmt`" object can be used to transfer the query written in JAVA program to Database and after processing, the results will be returned back and processed in java program.

Step4:

- Execute a Query:
 - ✓ `stmt.executeUpdate(query);`
 - ✓ `stmt.executeQuery(query);`
 - ✓ `stmt.execute();`
 - ✓ We will use database queries.

Step5:

- Close a Connection:

```
✓ con.close();
```

Steps to PostgreSQL Database from Java Application:

1. Install PostgreSQL and JDBC Driver

- Install PostgreSQL: Download and install PostgreSQL from the [official PostgreSQL website](#). Make note of your database username, password, and port (the default is 5432).
- Download PostgreSQL JDBC Driver: Get the latest PostgreSQL JDBC driver from [PostgreSQL JDBC Driver download page](#). Download the .jar file (e.g., postgresql-42.2.x.jar).

2. Set Up Eclipse for PostgreSQL Database Connection

2.1 Create a New Java Project

1. Open Eclipse.
2. Go to File > New > Java Project.
3. Name the project (e.g., PostgreSQLProject), then click Finish.

2.2 Add PostgreSQL JDBC Driver to the Project

1. Right-click on your project (PostgreSQLProject) in the Project Explorer.
2. Select Build Path > Configure Build Path.
3. In the Libraries tab, click on Add External JARs.
4. Browse to the location where you downloaded the PostgreSQL JDBC driver (e.g., postgresql-42.2.x.jar) and add it.
5. Click Apply and Close.

3. Create a Java Class to Connect to the PostgreSQL Database

3.1 Create a Class File

1. Right-click the src folder in your project.
2. Select New > Class.
3. Name the class (e.g., PostgreSQLJDBC), and check the box for public static void main(String[] args).
4. Click Finish.

3.2 Write Code to Connect to PostgreSQL

```

1*import java.sql.Connection;
5
6 public class DemoConnection {
7
8     public static void main(String[] args) throws SQLException {
9         String url="jdbc:postgresql://localhost:5432/employee";
10        String username="postgres";
11        String password="password";
12
13        Connection connection=DriverManager.getConnection(url,username,password);
14        if(connection!=null)
15        {
16            System.out.println("Connected successfully to PostgreSQL's Employee DB");
17        }
18        else
19        {
20            System.out.println("Fail to connect...");
21        }
22
23        connection.close();
24        System.out.println("Connection closed....");
25    }
26 }

```

CRUD Operations in PostgreSQL using Java:

```

1*import java.sql.Connection;
2 import java.sql.DriverManager;
3 import java.sql.ResultSet;
4 import java.sql.SQLException;
5 import java.sql.Statement;
6
7 public class CRUD_OperationsDemo {
8
9     public static void main(String[] args) throws SQLException {
10        String url="jdbc:postgresql://localhost:5432/employee";
11        String username="postgres";
12        String password="password";
13
14        Connection connection=DriverManager.getConnection(url,username,password);
15        if(connection!=null)
16        {
17            System.out.println("Connected successfully to PostgreSQL's Employee DB");
18        }
19        else
20        {
21            System.out.println("Fail to connect...");
22        }

```

```

24 //CRUD(Create, Retrieve, Update, Delete Operations
25 // Create table
26 String createQuery = "Create table cse(eid int, name char(10), salary real)";
27 Statement createStmt = connection.createStatement();
28 createStmt.executeUpdate(createQuery);
29 System.out.println("Table created Successfully...");
30
31 // Insert record
32 String insertQuery="insert into cse(eid,name,salary) values(1,'Azmal',2000),(2,'Mani',8500),(3,'Deepa',4500)";
33 Statement insertStmt=connection.createStatement();
34 insertStmt.executeUpdate(insertQuery);
35 System.out.println("Inserted record successfully...");
36
37 // Retrieve data - Select Query
38 Statement selectStmt=connection.createStatement();
39 ResultSet rs = selectStmt.executeQuery( "SELECT * FROM cse;" );
40 while ( rs.next() ) {
41     int eid = rs.getInt("eid");
42     String name = rs.getString("name");
43     float salary = rs.getFloat("salary");
44     System.out.println( "ID = " + eid );
45     System.out.println( "NAME = " + name );
46     System.out.println( "SALARY = " + salary );
47     System.out.println();
48 }
49
50 // Update record
51 String updateQuery="Update cse set salary = 3500 where eid = 1";
52 Statement updateStmt=connection.createStatement();
53 updateStmt.executeUpdate(updateQuery);
54 System.out.println("Record updated successfully...");
55
56 // Delete record
57 String deleteQuery="Delete from cse where eid = 1";
58 Statement deleteStmt=connection.createStatement();
59 deleteStmt.executeUpdate(deleteQuery);
60 System.out.println("Record deleted successfully...");
61
62 connection.close();
63 System.out.println("Connection closed...");
64 }

```

What is the difference between execute(), executeQuery() and executeUpdate() methods in JDBC?

1. execute()
2. execute Query()
3. execute Update()

The execute() method: This method is used to execute SQL DDL statements, it returns a boolean value specifying whether the ResultSet object can be retrieved.

```

public class Example {
    public static void main(String args[]) throws SQLException {
        //Registering the Driver
        DriverManager.registerDriver(new com.mysql.jdbc.Driver());
        //Getting the connection
        String mysqlUrl = "jdbc:mysql://localhost/sampleDB";
        Connection con = DriverManager.getConnection(mysqlUrl, "root", "password");
        System.out.println("Connection established.....");
        //Creating the Statement
        Statement stmt = con.createStatement();

        //Executing the statement
        String createTable = "CREATE TABLE Employee( "
            + "Name VARCHAR(255), "
            + "Salary INT NOT NULL, "
            + "Location VARCHAR(255))";
        boolean bool = stmt.execute(createTable);

        System.out.println(bool);
    }
}

```

executeUpdate(): This method is used to execute statements such as insert, update, delete. It returns an integer value representing the number of rows affected.

```
public class ExecuteUpdateExample {
    public static void main(String args[]) throws SQLException {
        //Registering the Driver
        DriverManager.registerDriver(new com.mysql.jdbc.Driver());
        //Getting the connection
        String mysqlUrl = "jdbc:mysql://localhost/sampleDB";
        Connection con = DriverManager.getConnection(mysqlUrl, "root", "password");
        System.out.println("Connection established.....");
        //Creating the Statement
        Statement stmt = con.createStatement();

        String insertData = "INSERT INTO Employee("
            + "Name, Salary, Location) VALUES "
            + "('Amit', 30000, 'Hyderabad'), "
            + "('Kalyan', 40000, 'Vishakhapatnam'), "
            + "('Renuka', 50000, 'Delhi'), "
            + "('Archana', 15000, 'Mumbai')";

        int i = stmt.executeUpdate(insertData);
        System.out.println("Rows inserted: "+i);
    }
}
```

executeQuery(): This method is used to execute statements that returns tabular data (example select). It returns an object of the class ResultSet.

```
public class ExecuteQueryExample {
    public static void main(String args[]) throws SQLException {
        //Registering the Driver
        DriverManager.registerDriver(new com.mysql.jdbc.Driver());
        //Getting the connection
        String mysqlUrl = "jdbc:mysql://localhost/sampleDB";
        Connection con = DriverManager.getConnection(mysqlUrl, "root", "password");
        System.out.println("Connection established.....");
        //Creating the Statement
        Statement stmt = con.createStatement();

        //Retrieving data
        ResultSet rs = stmt.executeQuery("Select *from Employee");

        while(rs.next()) {
            System.out.print("Name: "+rs.getString("Name")+", ");
            System.out.print("Salary: "+rs.getInt("Salary")+", ");
            System.out.print("City: "+rs.getString("Location"));
            System.out.println();
        }
    }
}
```

Difference Between Statement Vs PreparedStatement Vs CallableStatement In Java:

JDBC API provides 3 different interfaces to execute the different types of SQL queries. They are,

- 1) **Statement** – Used to execute normal SQL queries
- 2) **PreparedStatement** – Used to execute dynamic or parameterized SQL queries.
- 3) **CallableStatement** – Used to execute the stored procedures.

1) Statement:

Statement interface is used to execute normal SQL queries. You can't pass the parameters to SQL query at run time using this interface. This interface is preferred over other two interfaces if you are executing a particular SQL query only once. The performance of this interface is also very less compared to other two interfaces. In most of time, Statement interface is used for DDL statements like CREATE, ALTER, DROP etc. For example,

```

1  //Creating The Statement Object
2
3  Statement stmt = con.createStatement();
4
5  //Executing The Statement
6
7  stmt.executeUpdate("CREATE TABLE STUDENT(ID NUMBER NOT NULL, NAME VARCHAR)");

```

2) PreparedStatement

PreparedStatement is used to execute dynamic or parameterized SQL queries. PreparedStatement extends Statement interface. You can pass the parameters to SQL query at run time using this interface. It is recommended to use PreparedStatement if you are executing a particular SQL query multiple time. It gives better performance than Statement interface. Because, PreparedStatement are precompiled and the query plan is created only once irrespective of how many times you are executing that query. This will save lots of time.

```

1  //Creating PreparedStatement object
2
3  PreparedStatement pstmt = con.prepareStatement("update STUDENT set NAME = ? where ID = ?");
4
5  //Setting values to place holders using setter methods of PreparedStatement object
6
7  pstmt.setString(1, "MyName"); //Assigns "MyName" to first place holder
8
9  pstmt.setInt(2, 111); //Assigns "111" to second place holder
10
11 //Executing PreparedStatement
12
13 pstmt.executeUpdate();

```

3) CallableStatement

CallableStatement is used to execute the stored procedures. CallableStatement extends PreparedStatement.

Using CallableStatement, you can pass 3 types of parameters to stored procedures.

They are: **IN** – used to pass the values to stored procedure, **OUT** – used to hold the result returned by the stored procedure and **IN OUT** – acts as both IN and OUT parameter. Before calling the stored procedure, you must register

OUT parameters using `registerOutParameter ()` method of `CallableStatement`. The performance of this interface is higher than the other two interfaces. Because, it calls the stored procedures which are already compiled and stored in the database server.

```

1 //Creating CallableStatement object
2
3 CallableStatement cstmt = con.prepareCall("{call anyProcedure(?, ?, ?)}");
4
5 //Use cstmt.setter() methods to pass IN parameters
6
7 //Use cstmt.registerOutParameter() method to register OUT parameters
8
9 //Executing the CallableStatement
10
11 cstmt.execute();
12
13 //Use cstmt.getter() methods to retrieve the result returned by the stored procedure

```

How to Handle SQLException in JDBC?

Java Database Connectivity (JDBC) serves as the backbone for Java applications when interacting with databases. While establishing connections and executing queries, we developers often encounter `SQLExceptions`, which are inevitable in the real world. Handling those exceptions is crucial in the development of applications. Now let's understand more about SQL Exception.

Understanding SQL Exception

Exception is a type of condition when a program encounters a problem in execution and quits with a problematic error message. In JDBC, **when the program has trouble with a data source, it throws `SQLException`.**

SQL Exception methods in Java:

METHOD NAME	DESCRIPTION
<code>getErrorCode()</code>	Gets the error number associated with the exception.
<code>getMessage()</code>	This method gets the JDBC driver's error message for
<code>getSQLState()</code>	Gets the XOPEN SQLstate string. This method can return null.
<code>getNextException()</code>	In the exception chain, gets the next Exception object
<code>printStackTrace()</code>	Prints the current exception, or throwable, and it's backtrace to a standard error stream.
<code>printStackTrace(PrintStream s)</code>	To the print the stream we have specified, prints this throwable and its backtrace.
<code>printStackTrace(PrintStream s)</code>	Prints this throwable and it's backtrace to the print writer you specify.

Example:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class SQLExceptionExample {
    public static void main(String[] args) {
        Connection connection = null;
        PreparedStatement preparedStatement = null;

        try {
            // Load the JDBC driver
            Class.forName("com.mysql.cj.jdbc.Driver");

            // establish a connection to the database
            connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/your_database", "username",
"password");

            // Your SQL query with an intentional error (table name is misspelled)
            String sql = "SELECT * FROM non_existent_table";

            // create a PreparedStatement
            preparedStatement = connection.prepareStatement(sql);

            // execute the query (this will throw an SQLException in case of an error)
            preparedStatement.executeQuery();
        } catch (SQLException | ClassNotFoundException e) {
            // handle the SQL exception
            handleSQLException(e);
        } finally {
            // close the resources in the finally block
            try {
                if (preparedStatement != null) {
                    preparedStatement.close();
                }
                if (connection != null) {
                    connection.close();
                }
            } catch (SQLException e) {
                handleSQLException(e);
            }
        }
    }
}

```

```

private static void handleSQLException(SQLException e) {
    // handle the SQL exception
    int errorCode = e.getErrorCode();
    String sqlState = e.getSQLState();
    String errorMessage = e.getMessage();

    System.out.println("SQL Error Code: " + errorCode);
    System.out.println("SQL State: " + sqlState);
    System.out.println("Error Message: " + errorMessage);

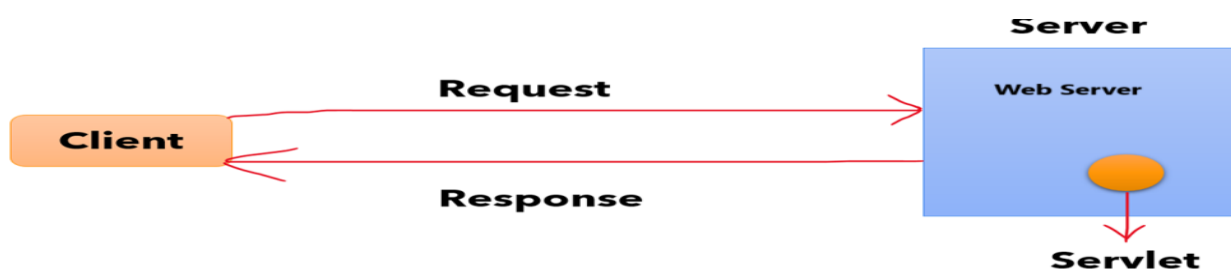
    e.printStackTrace();
}

```

Servlets in Java:

An Overview of Client-Server Communication and Servlet:

Client-server communication refers to the interaction between two entities: the client (usually a web browser or mobile app) and the server (which processes the client's requests and sends responses). The client and server communicate using the HTTP protocol, where the client sends a request to the server, and the server sends back a response.



Client-Server Communication Workflow:

Client sends a request:

- The client sends an HTTP request to the server.
- This request can be a **GET** request (to retrieve data) or a **POST** request (to send data to the server).

Server processes the request:

- The server receives the request and processes it, often by querying a database, performing some business logic, or generating a response dynamically.

Server sends a response:

- After processing, the server sends an HTTP response back to the client. This response can include content like HTML pages, JSON data, or files.

Client receives and renders the response:

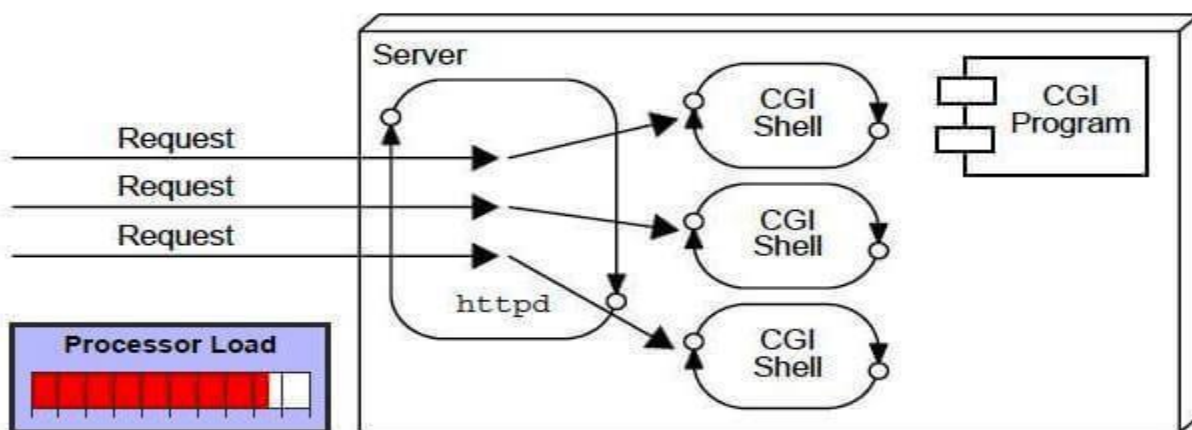
- The client receives the response and either renders the content (in the case of a web page) or processes the data (in the case of APIs or services).

What is a web application?

A web application is an application accessible from the web. A web application is composed of web components like Servlet, JSP, Filter, etc. and other elements such as HTML, CSS, and JavaScript. The web components typically execute in Web Server and respond to the HTTP request.

CGI (Common Gateway Interface)

CGI technology enables the web server to call an external program and pass HTTP request information to the external program to process the request. For each request, it starts a new process.

**Disadvantages of CGI**

There are many problems in CGI technology:

- If the number of clients increases, it takes more time for sending the response.
- For each request, it starts a process, and the web server is limited to start processes.
- It uses platform dependent language e.g. C, C++, perl.

What is a Servlet?

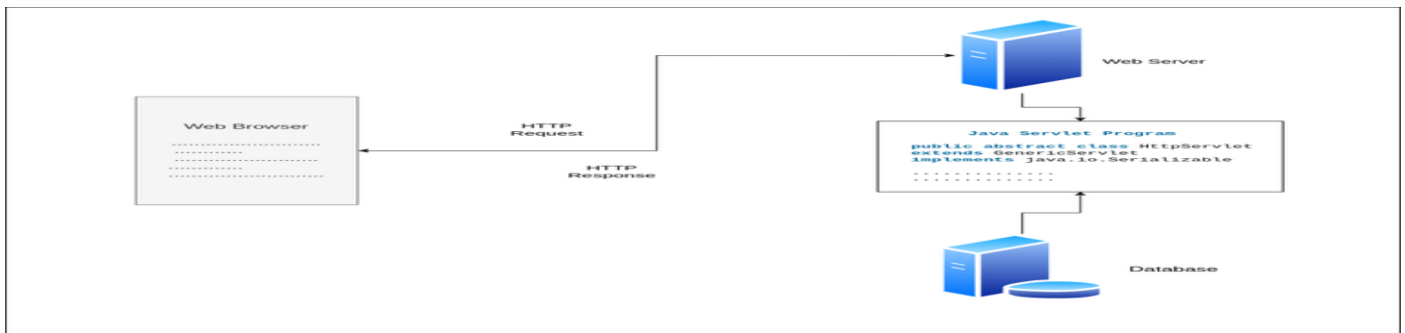
A servlet is a java file which takes request from client (Web browser), process it and return back the response.

A Servlet is a Java class used to extend the capabilities of a server by responding to incoming requests, typically from a web browser. Servlets run on Java-based web servers, such as Apache Tomcat, and are used to create dynamic web content. They are a key part of the Java EE (Enterprise Edition) platform and are often used to build web applications.

The properties of Servlets are as follows:

- Servlets work on the server side.
- Servlets are capable of handling complex requests obtained from the web server.

Java Servlets Architecture



Servlet Workflow:

Client Request(HTTP Request):

The client (browser) sends an HTTP request (GET, POST, etc.) to the web server.

Servlet Container:

The web server (like Tomcat) has a Servlet container that manages the lifecycle of servlets. This container is responsible for loading, initializing, and managing servlets. It also routes incoming requests to the appropriate servlet.

Servlet Processing:

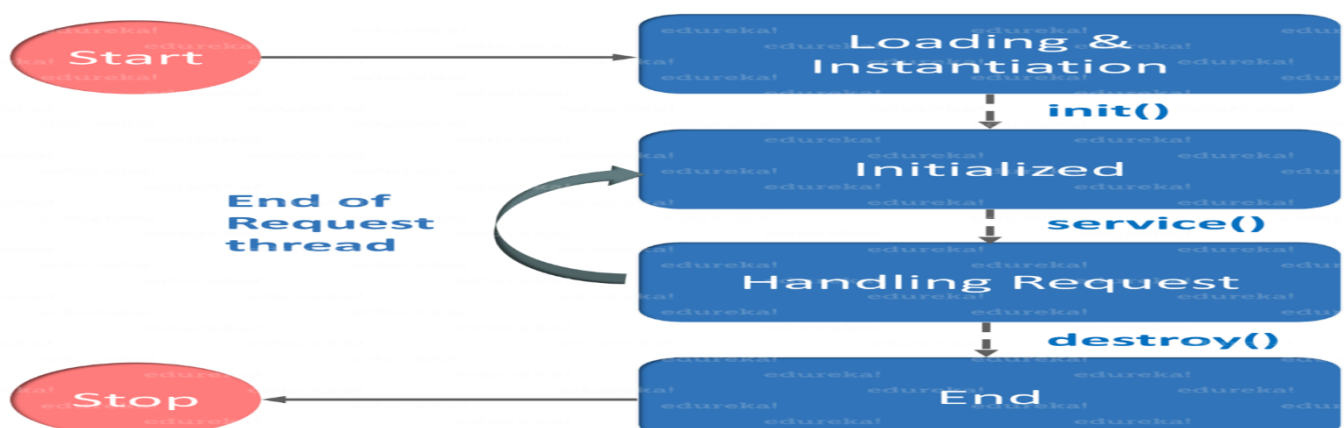
The servlet processes the client's request. It can read parameters from the request, interact with the database, perform business logic, or generate dynamic content (e.g., HTML, JSON).

Response(HTTP Response):

The servlet generates a response (usually an HTML page or data in JSON/XML format) and sends it back to the client via the web server.

Servlet Lifecycle:

A servlet follows a specific lifecycle:



init (): The servlet is initialized when the web server loads it for the first time, usually when the first request for that servlet is made.

```
public void init() throws ServletException {
    // Initialization code...
}
```

Service(): The servlet processes multiple client requests through the service() method, which calls doGet(), doPost(), etc., depending on the type of request.

The service() method is the main method to perform the actual task. The servlet container (i.e. web server) calls the service() method to handle requests coming from the client(browsers) and to write the formatted response back to the client.

```
public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException {
}
```

- public void doGet (HttpServletRequest res, HttpServletResponse res) throws ServletException {...}
- public void doPost (HttpServletRequest res, HttpServletResponse res) throws ServletException {...}

destroy (): The servlet is destroyed by the server when it is no longer needed, and resources are freed.

```
public void destroy() {
    // Finalization code...
}
```

Components of Servlets:

1. HttpServlet Class:

- The HttpServlet class is the base class for servlets that handle HTTP requests. It provides methods such as doGet() and doPost() for handling GET and POST requests, respectively.

2. Request and Response Objects:

- **HttpServletRequest:** Represents the client request. It allows access to request parameters, headers, cookies, etc.
- **HttpServletResponse:** Represents the server's response. It is used to write data back to the client.

3. Deployment Descriptor (web.xml):

- The web.xml file is used to configure servlets and map them to specific URLs. Although modern applications often use annotations (like @WebServlet) to configure servlets, web.xml is still useful for defining global settings.

Example of web.xml:

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app>
3   <display-name>servletDemoProject</display-name>
4
5   <servlet>
6     <servlet-name>servlet-1</servlet-name>
7     <servlet-class>servletDemoProject.MyServlet</servlet-class>
8   </servlet>
9
10  <servlet-mapping>
11    <servlet-name>servlet-1</servlet-name>
12    <url-pattern>/add</url-pattern>
13  </servlet-mapping>
14
15
16  <servlet>
17    <servlet-name>servlet-2</servlet-name>
18    <servlet-class>servletDemoProject.MyServlet</servlet-class>
19  </servlet>
20
21  <servlet-mapping>
22    <servlet-name>servlet-2</servlet-name>
23    <url-pattern>/sub</url-pattern>
24  </servlet-mapping>
25
26 </web-app>

```

Servlet Project Setup:

To set up a **Servlet** in **Eclipse** for web development, you typically use **Apache Tomcat** as the web server, along with the **Eclipse IDE** for Java EE development. Below are the steps to set up a servlet and configure it in Eclipse.

Prerequisites:

1. **Eclipse IDE for Java EE Developers** (Download from Eclipse).
2. **Apache Tomcat Server** (Download from [Tomcat](https://tomcat.apache.org/download-90.cgi)).
3. **JDK** (Ensure you have the JDK installed).

Step-by-Step Setup of a Servlet in Eclipse

1. Install and configure Apache Tomcat in Eclipse.
2. Create a Dynamic Web Project in Eclipse.
3. Write your Servlet using the `doGet()` or `doPost()` methods.
4. Deploy the project and run it on Tomcat to view your servlet in action.

1. Install and Configure Apache Tomcat in Eclipse

1. **Download Apache Tomcat:**
 - Download the latest version of Tomcat from the [official website](https://tomcat.apache.org/download-90.cgi)
 - <https://tomcat.apache.org/download-90.cgi>
2. **Open Eclipse** and set up Tomcat as your server:
 - Open **Eclipse**.
 - Go to **Window > Preferences > Server > Runtime Environments**.
 - Click **Add** and select the appropriate **Apache Tomcat** version (e.g., Tomcat 9.x).
 - Click **Next**, and browse to the directory where Tomcat is installed.
 - Click **Finish** to complete the setup.

2. Create a Dynamic Web Project

1. Create a New Dynamic Web Project:

- Go to **File > New > Project**.
- Select **Dynamic Web Project** under the **Web** category.
- Click **Next** and give your project a name (e.g., ServletDemo).
- Make sure the **Target Runtime** is set to **Apache Tomcat**.
- Leave other settings as default and click **Finish**.

2. Configure Web.xml (Optional):

- If prompted, choose to **Generate web.xml deployment descriptor**. This file is used for configuring servlets, though modern applications often use annotations instead.

3. Create a New Servlet

1. Create a New Servlet Class:

- In the **Project Explorer**, right-click the **src** folder of your project.
- Select **New > Servlet**.
- In the **Class Name**, give your servlet a name (e.g., HelloWorldServlet).
- Enter the **Java package name** (e.g., com.example.servlet).
- Click **Next**, and specify the URL mapping for the servlet (e.g., /hello).
- Click **Finish**.

2. Modify the Servlet Code:

- Modify the generated `doGet()` method to produce a simple HTML response. For example:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorldServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        // Set the response content type
        response.setContentType("text/html");

        // Write the response
        PrintWriter out = response.getWriter();
        out.println("<h1>Hello, World from Servlet!</h1>");
    }
}
```

4. Deploy and Run the Servlet

1. Deploy the Project to Tomcat:

- Right-click on your project (ServletDemo).
- Go to **Run As > Run on Server**.

- Select the **Tomcat Server** you set up earlier and click **Next**.
- Click **Finish** to deploy and start the server.

2. Access the Servlet:

- Once the server is running, open your web browser and navigate to the following URL:

```
http://localhost:8080/ServletDemo/hello
```

- You should see the output from your servlet, which is the Hello, World! message in HTML format.

5. (Optional) Configure web.xml for Servlet Mapping

While modern applications often use **annotations** (as we did above), you can still use the web.xml file for configuration.

1. Locate web.xml:

- In the **WebContent/WEB-INF** folder, you'll find web.xml.

2. Edit web.xml to manually configure the servlet:

```
xml

<servlet>
  <servlet-name>HelloWorldServlet</servlet-name>
  <servlet-class>com.example.servlet.HelloWorldServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>HelloWorldServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

Example of ServletRequest to display the name of the user:

In this example, we are displaying the name of the user in the servlet. For this purpose, we have used the `getParameter` method that returns the value for the given request parameter name.

index.html

```
<form action="welcome" method="get">
Enter your name<input type="text" name="name"><br>
<input type="submit" value="login">
</form>
```

DemoServ.java

```

import javax.servlet.http.*;
import javax.servlet.*;
import java.io.*;
public class DemoServ extends HttpServlet{
public void doGet(HttpServletRequest req,HttpServletResponse res)
throws ServletException,IOException
{
res.setContentType("text/html");
PrintWriter pw=res.getWriter();

String name=req.getParameter("name");//will return value
pw.println("Welcome "+name);

pw.close();
}}

```

Web.xml

```

1 <web-app>
2
3 <servlet>
4   <servlet-name>azmal</servlet-name>
5   <servlet-class>DemoServ</servlet-class>
6 </servlet>
7
8 <servlet-mapping>
9   <servlet-name>azmal</servlet-name>
10  <url-pattern>/welcome</url-pattern>
11 </servlet-mapping>
12
13 </web-app>

```

RequestDispatcher in Servlet:

The RequestDispatcher interface provides the facility of dispatching the request to another resource it may be html, servlet or jsp. This interface can also be used to include the content of another resource also. It is one of the way of servlet collaboration.

There are two methods defined in the RequestDispatcher interface.

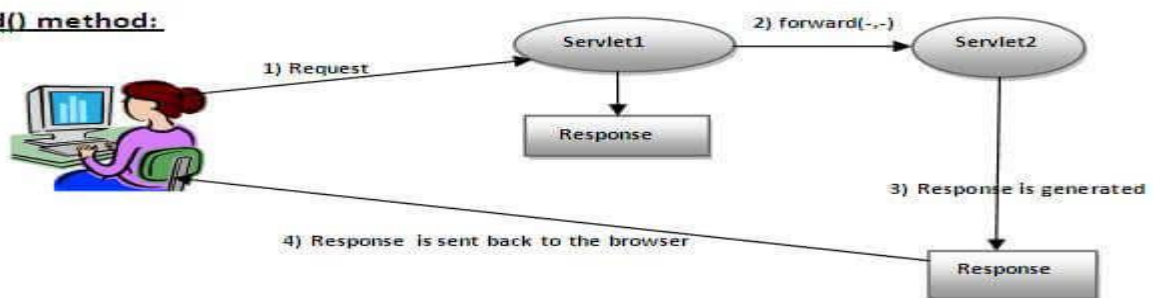
Methods of RequestDispatcher interface

The RequestDispatcher interface provides two methods. They are:

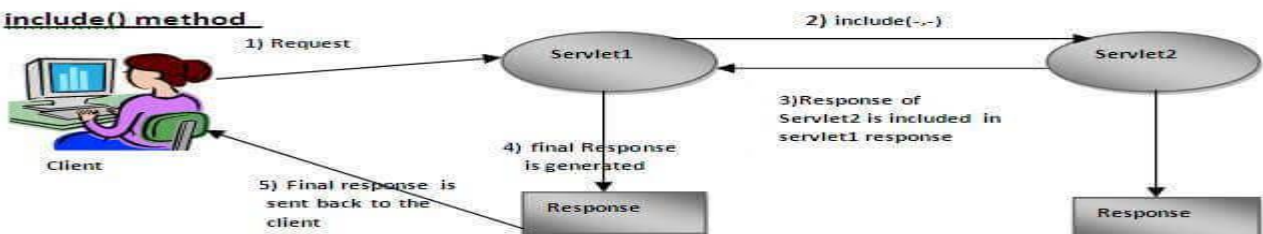
1. **public void forward(ServletRequest request,ServletResponse response)throws ServletException,java.io.IOException:** Forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.

2. **public void include (ServletRequest request, ServletResponse response) throws ServletException, java.io.IOException:** Includes the content of a resource (servlet, JSP page, or HTML file) in the response.

forward() method:



include() method



Syntax of getRequestDispatcher method

```
public RequestDispatcher getRequestDispatcher(String resource);
```

Example of using getRequestDispatcher method

```
RequestDispatcher rd=request.getRequestDispatcher("servlet2");
//servlet2 is the url-pattern of the second servlet

rd.forward(request, response);//method may be include or forward
```

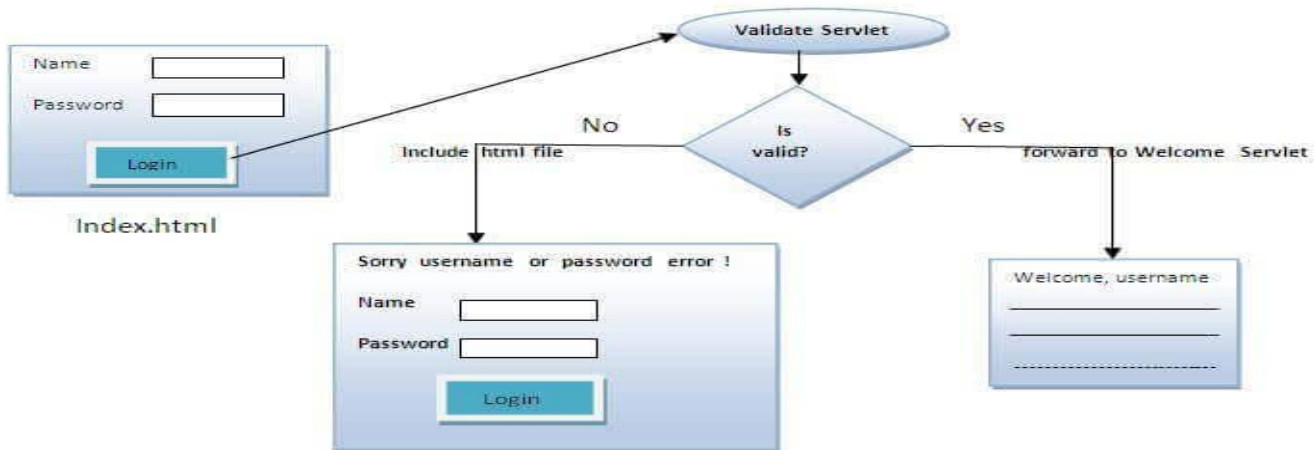
UseCase:

Example of RequestDispatcher interface:

In this example, we are validating the password entered by the user. If password is servlet, it will forward the request to the WelcomeServlet, otherwise will show an error message: sorry username or password error!. In this program,

we are checking for hardcoded information. But you can check it to the database also that we will see in the development chapter. In this example, we have created following files

- **index.html file:** for getting input from the user.
- **Login.java file:** a servlet class for processing the response. If password is servet, it will forward the request to the welcome servlet.
- **WelcomeServlet.java file:** a servlet class for displaying the welcome message.
- **web.xml file:** a deployment descriptor file that contains the information about the servlet.



index.html

```
<form action="servlet1" method="post">
Name:<input type="text" name="userName"/><br/>
Password:<input type="password" name="userPass"/><br/>
<input type="submit" value="login"/>
</form>
```

Login.java

```

12 public class Login extends HttpServlet {
13     public void doPost(HttpServletRequest request, HttpServletResponse response)
14         throws ServletException, IOException {
15
16         response.setContentType("text/html");
17         PrintWriter out = response.getWriter();
18
19         String n=request.getParameter("userName");
20         String p=request.getParameter("userPass");
21
22         if(p.equals("servlet")){
23             RequestDispatcher rd=request.getRequestDispatcher("servlet2");
24             rd.forward(request, response);
25         }
26         else{
27             out.print("Sorry UserName or Password Error!");
28             RequestDispatcher rd=request.getRequestDispatcher("/index.html");
29             rd.include(request, response);
30         }
31     }
32 }

```

WelcomeServlet.java

```

12 public class WelcomeServlet extends HttpServlet {
13     public void doPost(HttpServletRequest request, HttpServletResponse response)
14         throws ServletException, IOException {
15
16         response.setContentType("text/html");
17         PrintWriter out = response.getWriter();
18
19         String n=request.getParameter("userName");
20         out.print("Welcome "+n);
21     }
22 }

```

Web.xml

```

2 <web-app>
3     <servlet>
4         <servlet-name>Login</servlet-name>
5         <servlet-class>Login</servlet-class>
6     </servlet>
7     <servlet>
8         <servlet-name>WelcomeServlet</servlet-name>
9         <servlet-class>WelcomeServlet</servlet-class>
10    </servlet>
11
12    <servlet-mapping>
13        <servlet-name>Login</servlet-name>
14        <url-pattern>/servlet1</url-pattern>
15    </servlet-mapping>
16    <servlet-mapping>
17        <servlet-name>WelcomeServlet</servlet-name>
18        <url-pattern>/servlet2</url-pattern>
19    </servlet-mapping>
20
21    <welcome-file-list>
22        <welcome-file>index.html</welcome-file>
23    </welcome-file-list>
24
25 </web-app>

```

SendRedirect in servlet:

- The **sendRedirect ()** method of HttpServletResponse interface can be used to redirect response to another resource, it may be servlet, jsp or html file.
- It accepts relative as well as absolute URL.
- It works at client side because it uses the URL bar of the browser to make another request. So, it can work inside and outside the server.

Difference between forward() and sendRedirect() method

There are many differences between the forward() method of RequestDispatcher and sendRedirect() method of HttpServletResponse interface. They are given below:

forward() method	sendRedirect() method
The forward() method works at server side.	The sendRedirect() method works at client side.
It sends the same request and response objects to another servlet.	It always sends a new request.
It can work within the server only.	It can be used within and outside the server.
Example: request.getRequestDispatcher("servlet2").forward(request,response);	Example: response.sendRedirect("servlet2");

Syntax of sendRedirect() method:

public void sendRedirect(String URL)throws IOException;

Example of sendRedirect() method:

response.sendRedirect("http://www.kluniversity.in");

Example of sendRedirect method in servlet:

In this example, we are redirecting the request to the google server. Notice that sendRedirect method works at client side, that is why we can our request to anywhere. We can send our request within and outside the server.

DemoServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class DemoServlet extends HttpServlet{
    public void doGet(HttpServletRequest req,HttpServletResponse res)
    throws ServletException,IOException
    {
        res.setContentType("text/html");
        PrintWriter pw=res.getWriter();

        response.sendRedirect("http://www.google.com");

        pw.close();
    }
}
```

Creating custom google search using sendRedirect:

In this example, we are using `sendRedirect` method to send request to google server with the request data.

index.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>sendRedirect example</title>
</head>
<body>

<form action="MySearcher">
<input type="text" name="name">
<input type="submit" value="Google Search">
</form>

</body>
</html>
```

MySearcher.java

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

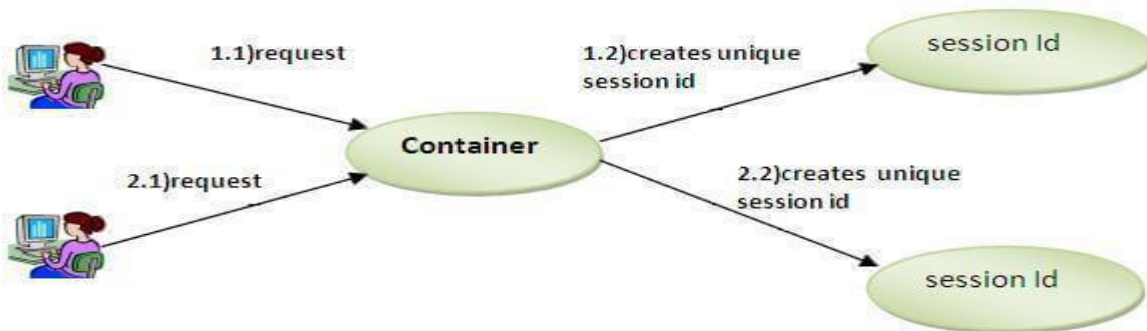
public class MySearcher extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String name=request.getParameter("name");
        response.sendRedirect("https://www.google.co.in/#q="+name);
    }
}
```

HttpSession interface:

In such case, container creates a session id for each user. The container uses this id to identify the particular user. An object of `HttpSession` can be used to perform two tasks:

1. bind objects
2. view and manipulate information about a session, such as the session identifier, creation time, and last accessed time.



How to get the HttpSession object ?

The HttpServletRequest interface provides two methods to get the object of HttpSession:

1. **public HttpSession getSession():**Returns the current session associated with this request, or if the request does not have a session, creates one.
2. **public HttpSession getSession(boolean create):**Returns the current HttpSession associated with this request or, if there is no current session and create is true, returns a new session.

Commonly used methods of HttpSession interface

1. **public String getId():**Returns a string containing the unique identifier value.
2. **public long getCreationTime():**Returns the time when this session was created, measured in milliseconds since midnight January 1, 1970 GMT.
3. **public long getLastAccessedTime():**Returns the last time the client sent a request associated with this session, as the number of milliseconds since midnight January 1, 1970 GMT.
4. **public void invalidate():**Invalidates this session then unbinds any objects bound to it.

Example of using HttpSession

In this example, we are setting the attribute in the session scope in one servlet and getting that value from the session scope in another servlet. To set the attribute in the session scope, we have used the `setAttribute()` method of HttpSession interface and to get the attribute, we have used the `getAttribute` method.

index.html

```

<form action="servlet1">
Name:<input type="text" name="userName"/><br/>
<input type="submit" value="go"/>
</form>
  
```

FirstServlet.java

```

public class FirstServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response){
        try{

            response.setContentType("text/html");
            PrintWriter out = response.getWriter();

            String n=request.getParameter("userName");
            out.print("Welcome "+n);

            HttpSession session=request.getSession();
            session.setAttribute("uname",n);

            out.print("<a href='servlet2'>visit</a>");

            out.close();

        }catch(Exception e){System.out.println(e);}
    }

```

SecondServlet.java

```

public class SecondServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
    try{

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session=request.getSession(false);
        String n=(String)session.getAttribute("uname");
        out.print("Hello "+n);

        out.close();

    }catch(Exception e){System.out.println(e);}
    }

```

web.xml

```
<web-app>

<servlet>
<servlet-name>s1</servlet-name>
<servlet-class>FirstServlet</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>s1</servlet-name>
<url-pattern>/servlet1</url-pattern>
</servlet-mapping>

<servlet>
<servlet-name>s2</servlet-name>
<servlet-class>SecondServlet</servlet-class>
</servlet>

<servlet-mapping>
<servlet-name>s2</servlet-name>
<url-pattern>/servlet2</url-pattern>
</servlet-mapping>

</web-app>
```

JSP & Advantages over servlets, features, syntax, Lifecycle of JSP:

- Java Server Pages (JSP) is a technology which is used to develop web pages by inserting [Java](#) code into the HTML pages by making special JSP tags. The JSP tags which allow java code to be included into it are `<% —java code—%>`.
- It can consist of either HTML or XML (combination of both is also possible) with JSP actions and commands.
- It can be used as HTML page, which can be used in forms and registration pages with the dynamic content into it.
- Dynamic content includes some fields like dropdown, checkboxes, etc. whose value will be fetched from the database.
- This can also be used to access JavaBeans objects.
- We can share information across pages using request and response objects.
- JSP can be used for separation of the view layer with the business logic in the web application.

Advantages of JSP over Servlet:

There are many advantages of JSP over the Servlet. They are as follows:

1) Extension to Servlet

JSP technology is the extension to Servlet technology. We can use all the features of the Servlet in JSP. In addition to, we can use implicit objects, predefined tags, expression language and Custom tags in JSP, that makes JSP development easy.

2) Easy to maintain

JSP can be easily managed because we can easily separate our business logic with presentation logic. In Servlet technology, we mix our business logic with the presentation logic.

3) Fast Development: No need to recompile and redeploy

If JSP page is modified, we don't need to recompile and redeploy the project. The Servlet code needs to be updated and recompiled if we have to change the look and feel of the application.

4) Less code than Servlet

In JSP, we can use many tags such as action tags, JSTL, custom tags, etc. that reduces the code. Moreover, we can use EL, implicit objects, etc.

Features of JSP

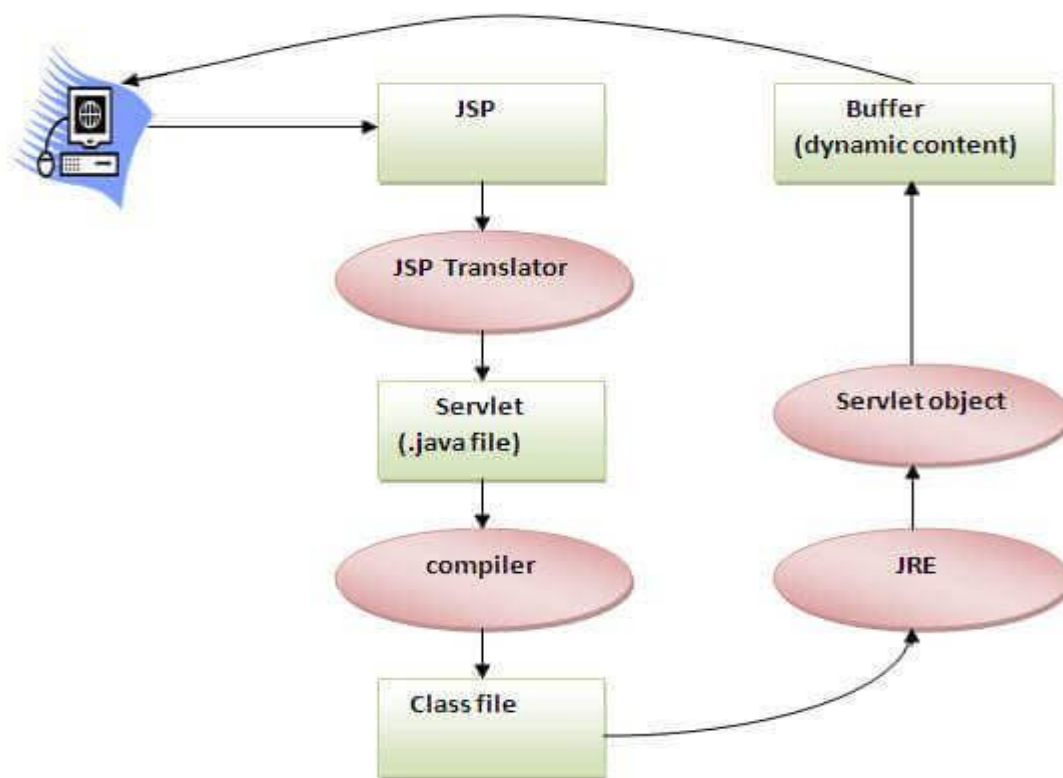
1. **Custom Tags and Tag Libraries:** JSP allows the use of custom tags and tag libraries (JSTL) for modular, reusable components.
2. **Expression Language (EL):** EL provides a simple way to access data stored in JavaBeans, request parameters, and other objects without using Java code.
3. **Built-in Objects:** JSP provides implicit objects like out, request, response, session, etc., for easier coding.
4. **Error Handling:** JSP supports error handling through specific pages, which can be customized for error scenarios.
5. **Directives and Actions:** JSP supports directives (e.g., page, include, taglib) and actions (e.g., jsp:useBean, jsp:setProperty) to control page attributes, include other resources, and interact with JavaBeans.
6. **Platform Independence:** As a Java-based technology, JSPs are platform-independent, supporting deployment on any platform with a compatible servlet container (e.g., Tomcat, JBoss).

The Lifecycle of a JSP Page:

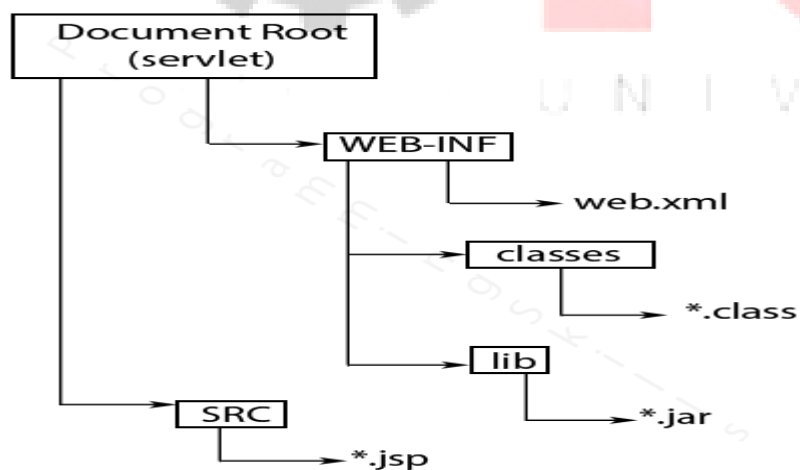
The JSP pages follow these phases:

- Translation of JSP Page
- Compilation of JSP Page
- Classloading (the classloader loads class file)
- Instantiation (Object of the Generated Servlet is created).
- Initialization (the container invokes jsplnit() method).
- Request processing (the container invokes _jspService() method).
- Destroy (the container invokes jspDestroy() method).

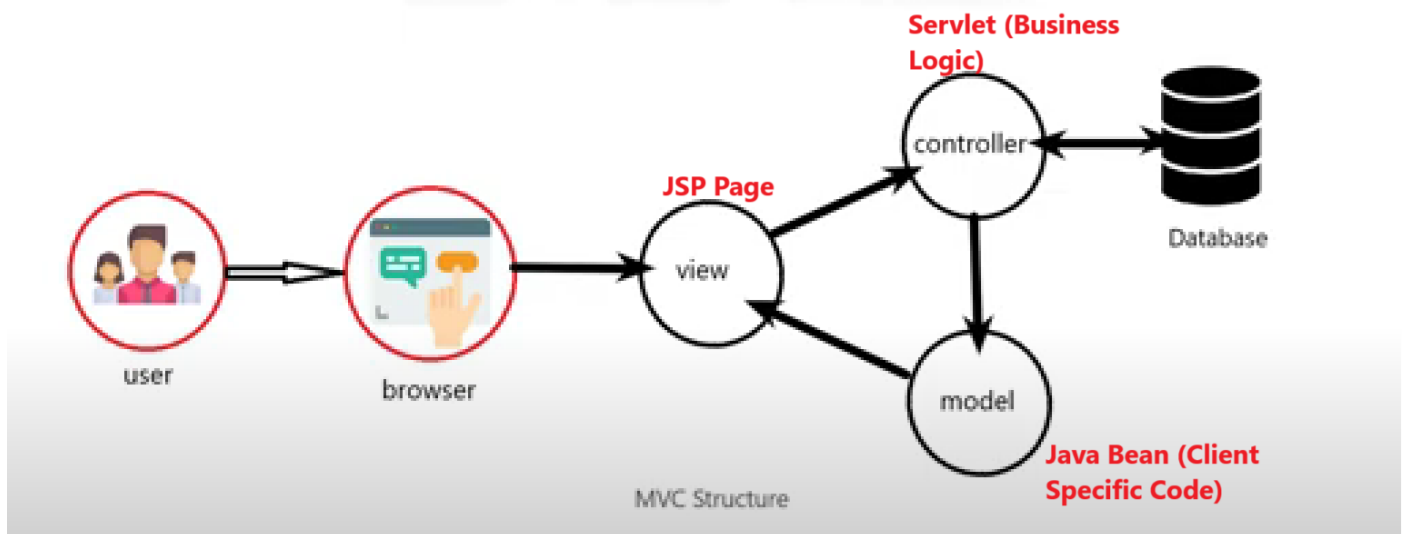
jsplnit(), _jspService() and jspDestroy() are the life cycle methods of JSP.



In order to execute any JSP program one must follow the following directory structure:



How JSP Works?



Example of a JSP Page

Here's a simple example of a JSP page displaying a message to the user and using JSP syntax elements.

```
<%@ page language="java" contentType="text/html" %>
<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>
    <h1>Welcome to JSP Example</h1>

    <!-- Declaration -->
    <%! String message = "Hello, JSP!"; %>

    <!-- Scriptlet to set a new message -->
    <% message = "Hello, User!"; %>

    <p>Message: <%= message %></p>

    <!-- Using an action to interact with a bean -->
    <jsp:useBean id="user" class="com.example.User" />
    <jsp:setProperty name="user" property="name" value="John" />
    <p>User Name: <jsp:getProperty name="user" property="name" /></p>
</body>
</html>
```

JSP Tags:

Syntax/Tags	Descriptions
<code><% scriptlets %></code>	used for multiple lines of java code
<code><%= expression %></code>	used for few or small java code
<code><%! declaration %></code>	used for declare java variable or method
<code><%@ directive %></code>	used for declare as like global variable that affect the whole jsp page
<code><%-- comment --%></code>	used for make comment that ignore by browser

1. JSP Declaration
2. JSP Scriptlet
3. JSP Expression
4. JSP Directive
5. JSP Comments

1.JSP Declaration

A declaration tag is a piece of Java code for declaring variables, methods and classes. If we declare a variable or method inside declaration tag it means that the declaration is made inside the servlet class but outside the service method.

Syntax of declaration tag:

```
<%! Dec var %>
```

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="UTF-8">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <%! int count =10; %>
11 <% out.println("The Number is " +count); %>
12 </body>
13 </html>

```

2.JSP Scriptlet

- Scriptlet tag allows to write Java code into JSP file.
- JSP container moves statements in `_jspService()` method while generating servlet from jsp.
- For each request of the client, service method of the JSP gets invoked hence the code inside the Scriptlet executes for every request.
- A Scriptlet contains java code that is executed every time JSP is invoked.

Syntax of Scriptlet tag:

```
<% java code %>
```

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="UTF-8">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <% int num1=10;
11    int num2=40;
12    int num3 = num1+num2;
13    out.println("Scriptlet Number is " +num3);
14 %>
15 </body>
16 </html>

```

3. JSP Expression

- Expression tag evaluates the expression placed in it.
- It accesses the data stored in stored application.
- It allows create expressions like arithmetic and logical.
- It produces scriptless JSP page.

Syntax:

```
<%= expression %>
```

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="UTF-8">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <% out.println("The expression number is "); %>
11 <% int num1=10; int num2=10; int num3 = 20; %>
12 <%= num1*num2+num3 %>
13 </body>
14 </html>

```

JSP Comments

- Comments are the one when JSP container wants to ignore certain texts and statements.
- When we want to hide certain content, then we can add that to the comments section.

Syntax:

```
<% -- JSP Comments %>
```

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2     pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6 <meta charset="UTF-8">
7 <title>Insert title here</title>
8 </head>
9 <body>
10 <!-- type your comments here -->
11 <% out.println("This is comments example"); %>
12 </body>
13 </html>

```

Note: Shortcut: ctrl+shift+/ for comment and ctrl+shift+\ for remove comments

Creating a simple JSP Page

- A JSP page has an HTML body incorporated with Java code into it
- We are creating a simple JSP page which includes declarations, scriptlets, expressions, comments tags in it.

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"
2   pageEncoding="UTF-8"%>
3 <!DOCTYPE html>
4 <html>
5 <head>
6   <meta charset="UTF-8">
7   <title>Insert title here</title>
8 </head>
9 <body>
10 <!-- This is a JSP example with scriptlets, comments , expressions --%>
11 <% out.println("This is Demo JSP Example"); %>
12 <% out.println("The number is "); %>
13 <%! int num12 = 12; int num32 = 12; %>
14 <%= num12*num32 %>
15 Today's date: <%= (new java.util.Date()).toLocaleString()%>
16 </body>
17 </html>

```

What are JSP Directives?

- JSP directives are the messages to JSP container. They provide global information about an entire JSP page.
- JSP directives are used to give special instruction to a container for translation of JSP to servlet code.
- In JSP life cycle phase, JSP has to be converted to a servlet which is the translation phase.
- They give instructions to the container on how to handle certain aspects of JSP processing
- Directives can have many attributes by comma separated as key-value pairs.
- In JSP, directive is described in <%@ %> tags.

Syntax of Directive:

```
<%@ directive attribute="" %>
```

Types of Directives in JSP

There are three types of directives in JSP:

1. Page directive
2. Include directive
3. Taglib directive

1) JSP Page directive

Syntax of Page directive:

```
<%@ page...%>
```

- It provides attributes that get applied to entire JSP page.
- It defines page dependent attributes, such as scripting language, error page, and buffering requirements.

- It is used to provide instructions to a container that pertains to current JSP page.

Following are its list of attributes associated with page directive:

1. Language
2. Extends
3. Import
4. contentType
5. info
6. session
7. isThreadSafe
8. autoflush
9. buffer
10. isErrorPage
11. pageEncoding
12. errorPage
13. isELIgnored

Example:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
import="java.util.Date" pageEncoding="ISO-8859-1"%>
```

10) isErrorPage:

- It indicates that JSP Page that has an errorPage will be checked in another JSP page
- Any JSP file declared with “isErrorPage” attribute is then capable to receive exceptions from other JSP pages which have error pages.
- Exceptions are available to these pages only.
- The default value is false.

Syntax of isErrorPage:

```
<%@ page isErrorPage="true/false"%>
```

12) errorPage:

This attribute is used to set the error page for the JSP page if JSP throws an exception and then it redirects to the exception page.

Syntax of errorPage:

```
<%@ page errorPage="value" %>
```

Example with four attributes:

```

1 <%@ page language="java" contentType="text/html; isELIgnored="false"%
2     pageEncoding="UTF-8"%>
3 <%@page import="java.util.Date" %>
4
5 <!DOCTYPE html>
6 <html>
7 <head>
8 <meta charset="UTF-8">
9 <title>Insert title here</title>
10 </head>
11 <body>
12 <h1>Welcome to JSP</h1>
13 <a>Date is:</a>
14 <%= new java.util.Date() %>
15 </body>
16 </html>

```

2)JSP Include directive

- JSP “include directive” (codeline 8) is used to include one file to the another file
- This included file can be HTML, JSP, text files, etc.
- It is also useful in creating templates with the user views and break the pages into header&footer and sidebar actions.
- It includes file during translation phase

Syntax of include directive:

```
<%@ include....%>
```

Example:

Directive_jsp2.jsp (Main file)

```

1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <%@ include file="directive_header_jsp3.jsp"%>
4 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://ww
5 <html>
6 <head>
7 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
8 <title>title here</title>
9 </head>
10 <body>
11   <a>This is the main file</a>
12 </body>
13 </html>

```

Directive_header_jsp3.jsp (which is included in the main file)

```

1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3 <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.
4 <html>
5 <head>
6 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
7
8 </head>
9 <body>
10 <a>Header file : </a>
11 <%int count =1; count++;
12 out.println(count);%> :
13 </body>
14 </html>

```

3) JSP Taglib Directive

- JSP taglib directive is used to define the tag library with “taglib” as the prefix, which we can use in [JSP](#).
- JSP taglib directive is used in the JSP pages using the JSP standard tag libraries
- It uses a set of custom tags, identifies the location of the library and provides means of identifying custom tags in JSP page.

Syntax of taglib directive:

```
<%@ taglib uri="uri" prefix="value"%>
```

Example:

```

1 <%@ page language="java" contentType="text/html; charset=ISO-8859-1"
2   pageEncoding="ISO-8859-1"%>
3   <%@ taglib prefix="klutag" uri="http://java.sun.com/jsp/jstl/core" %>
4 <!DOCTYPE html PUBLIC "-//KLU//DTD HTML 4.01 Transitional//EN" "http://www.
5 <html>
6 <head>
7 <meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
8 <title>title here</title>
9 <klutag:hello/>
10 </head>
11 <body>
12 </body>
13 </html>

```

Servlet demo application development with JSP form data:

Steps for the Demo Application

1. Set up the Project: Create a new Java web project.
2. Create JSP Form Page: A form in a JSP file for collecting user input.
3. Develop the Servlet: A servlet to handle the request and response.
4. Display Results: Use a JSP to display the results.

Step 1: Set up the Project

- Create a new Java Web project in your IDE.
- Set up your project to use Apache Tomcat or any other server that supports Java Servlets and JSP.
- Make sure the project structure includes WEB-INF and META-INF folders and is configured as a dynamic web project.

Step 2: Create the JSP Form Page

- Create a JSP page named `form.jsp` inside the `webapp` or `WebContent` directory:

```

<!-- form.jsp -->
<!DOCTYPE html>
<html>
<head>
    <title>User Form</title>
</head>
<body>
    <h2>User Registration Form</h2>
    <form action="ProcessForm" method="post">
        Name: <input type="text" name="name"><br><br>
        Email: <input type="email" name="email"><br><br>
        <input type="submit" value="Submit">
    </form>
</body>
</html>

```

Step 3: Create the Servlet to Handle Form Data

- Create a servlet named SubmitServlet.java
- This servlet will process the form data and forward the response to another JSP page for display.

```

3 //SubmitServlet.java
4*import java.io.IOException;
10
11 @WebServlet("/SubmitServlet")
12 public class SubmitServlet extends HttpServlet {
13     protected void doPost(HttpServletRequest request, HttpServletResponse response)
14         throws ServletException, IOException {
15         // Retrieve data from form
16         String name = request.getParameter("name");
17         String email = request.getParameter("email");
18
19         // Set data as request attributes
20         request.setAttribute("name", name);
21         request.setAttribute("email", email);
22
23         // Forward data to result.jsp
24         request.getRequestDispatcher("result.jsp").forward(request, response);
25     }
26 }

```

Step 4: Create the JSP to Display the Results

- Create a result.jsp file to show the processed data to the user.

```

<!-- result.jsp -->
<!DOCTYPE html>
<html>
<head>
    <title>Form Results</title>
</head>
<body>
    <h2>Submitted Information</h2>
    <p><strong>Name:</strong> ${name}</p>
    <p><strong>Email:</strong> ${email}</p>
</body>
</html>

```

Step 5: Configure web.xml (Optional)

- If you aren't using annotations in the servlet, configure the servlet mapping in web.xml:

```

<!-- web.xml -->
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    version="3.1">

    <servlet>
        <servlet-name>ProcessFormServlet</servlet-name>
        <servlet-class>ProcessFormServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>ProcessFormServlet</servlet-name>
        <url-pattern>/ProcessForm</url-pattern>
    </servlet-mapping>
</web-app>

```

Project Flow:

1. **form.jsp**: The user fills in the form and submits it.
2. **SubmitServlet**: Receives the form data and forwards it to result.jsp.
3. **result.jsp**: Displays the submitted name and email.

Servlet demo Application development with JDBC to persist data to the database:

(Servlet – JSP – JDBC)

Creating a servlet-based application with JDBC and PostgreSQL involves taking form data from a JSP page, processing it in a servlet, and then saving it to a PostgreSQL database. Below is a step-by-step guide to building this type of application.

Prerequisites

1. PostgreSQL installed and running.
2. A JDBC driver for PostgreSQL added to your project (postgresql-42.2.x.jar, which you can download from [PostgreSQL JDBC](#)).
3. An IDE like Eclipse or IntelliJ, and a servlet container like Apache Tomcat.

Step 1: Set Up the Database

1. Open your PostgreSQL client (e.g., pgAdmin or command-line client).
2. Create a new database and a table to store user data.

```
CREATE DATABASE demo_db;
```

```
\c demo_db
```

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);
```

Step 2: Configure Database Connection in Your Project

1. Add the PostgreSQL JDBC driver to your project's classpath.
2. Create a database utility class to handle the database connection.

Example:

```
3 import java.sql.Connection;
6
7 public class DatabaseUtil {
8     private static final String URL = "jdbc:postgresql://localhost:5432/usersdata";
9     private static final String USER = "postgres";
10    private static final String PASSWORD = "password";
11
12    public static Connection getConnection() throws SQLException {
13        return DriverManager.getConnection(URL, USER, PASSWORD);
14    }
15 }
```

Step 3: Create the JSP Form

1. In your project, create a JSP file named index.jsp.

2. Write a simple HTML form to collect user data.

```
<!-- index.jsp -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Servlet with JDBC and PostgreSQL</title>
</head>
<body>
    <h2>Enter Your Information</h2>
    <form action="SubmitServlet" method="post">
        <label for="name">Name:</label>
        <input type="text" id="name" name="name" required><br><br>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email" required><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

Step 4: Create a Servlet to Handle the Form Data and Persist It to the Database

1. Create a servlet named SubmitServlet.
2. This servlet will process form data, establish a database connection, and save the data.

Example:

```
14 @WebServlet("/SubmitServlet")
15 public class SubmitServlet extends HttpServlet {
16     protected void doPost(HttpServletRequest request, HttpServletResponse response)
17         throws ServletException, IOException {
18
19         String name = request.getParameter("name");
20         String email = request.getParameter("email");
21
22         try (Connection conn = DBUtil.getConnection()) {
23             String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
24             try (PreparedStatement stmt = conn.prepareStatement(sql)) {
25                 stmt.setString(1, name);
26                 stmt.setString(2, email);
27                 stmt.executeUpdate();
28             }
29
30             // Forward to a success page or display a success message
31             request.setAttribute("message", "Data successfully saved to the database.");
32             request.getRequestDispatcher("result.jsp").forward(request, response);
33
34         } catch (SQLException e) {
35             e.printStackTrace();
36             request.setAttribute("message", "Error: Could not save data to the database.");
37             request.getRequestDispatcher("result.jsp").forward(request, response);
38         }
39     }
40 }
```

In this servlet:

- We retrieve name and email from the form data using `request.getParameter()`.
- We then use a `PreparedStatement` to insert the data into the users table.
- Finally, we forward to `result.jsp` with a success or error message

Step 5: Create a JSP Page to Display the Result

1. Create a new JSP file named `result.jsp`.
2. Display the result of the database operation.

```
<!-- result.jsp -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Form Data Result</title>
</head>
<body>
  <h2>Form Submission Result</h2>
  <p>${message}</p>
  <a href="index.jsp">Back to Form</a>
</body>
</html>
```

Step 6: Deploy and Test the Application

1. Deploy the application on Tomcat.
2. Open `index.jsp` in your browser (e.g., http://localhost:8080/your_project_name/index.jsp).
3. Submit the form, and the data should be saved to the PostgreSQL database.
4. Verify the saved data by querying the users table in PostgreSQL:

```
SELECT * FROM users;
```

JSP Action Tags

JSP action tags provide dynamic functionality and interaction with JavaBeans, page forwarding, and more.

a. `<jsp:useBean>`

Creates or locates a JavaBean instance.

```
jsp
<jsp:useBean id="user" class="com.example.User" scope="session" />
```

b. `<jsp:setProperty>`

Sets a property value for a JavaBean.

```
jsp
<jsp:setProperty name="user" property="name" value="John Doe" />
```

c. `<jsp:getProperty>`

Retrieves a property value from a JavaBean and outputs it.

```
jsp
<p>User Name: <jsp:getProperty name="user" property="name" /></p>
```

d. `<jsp:include>`

Includes another resource at request time (dynamic inclusion).

```
jsp
<jsp:include page="footer.jsp" />
```

e. `<jsp:forward>`

Forwards the request to another resource.

```
jsp
<jsp:forward page="home.jsp" />
```

f. `<jsp:param>`

Passes parameters to an included or forwarded resource. Often used within `<jsp:include>` or `<jsp:forward>`.

```
jsp

<jsp:forward page="profile.jsp">
    <jsp:param name="userId" value="12345" />
</jsp:forward>
```

Java Beans:

In JSP (Java Server Pages), **JavaBeans** are reusable Java classes that encapsulate data and business logic, which can be easily accessed and manipulated from JSP pages. JavaBeans are primarily used to separate the data (or business logic) from the presentation layer, making the code cleaner, more modular, and easier to maintain.

Rule of JavaBeans:

1. **Encapsulation:** JavaBeans store data in private fields and provide public getter and setter methods to access and modify this data. This encapsulation helps protect the data and allows controlled access.
2. **No-Argument Constructor:** JavaBeans must have a public, no-argument constructor to allow easy instantiation.
3. **Serializable:** JavaBeans typically implement `java.io.Serializable`, making it possible to save the state of the bean if needed.

Why Use JavaBeans in JSP?

Using JavaBeans in JSP helps to:

- **Separate business logic from presentation logic:** By keeping data and business logic in JavaBeans, JSP pages can focus only on displaying content, improving maintainability.
- **Reuse code:** JavaBeans can be reused across different JSP pages or applications, reducing redundancy.
- **Simplify data access and manipulation:** With standardized getter and setter methods, accessing and modifying data in JSP becomes straightforward.

Example of Java Bean Class:

```
3 public class User {
4     private String name;
5     private String email;
6
7     public User() {
8         // No-argument constructor
9     }
10    public String getName() {
11        return name;
12    }
13    public void setName(String name) {
14        this.name = name;
15    }
16    public String getEmail() {
17        return email;
18    }
19    public void setEmail(String email) {
20        this.email = email;
21    }
22 }
```

Demo Application development on JSP Action tags:

Here's a simple JSP application demonstrating various JSP action tags. This application will show how to use action tags like `<jsp:useBean>`, `<jsp:setProperty>`, `<jsp:getProperty>`, `<jsp:include>`, and `<jsp:forward>`.

The application structure will be as follows:

1. **Home Page (index.jsp)**: Main page with a form to collect user data.
2. **JavaBean (User.java)**: A Java class to hold the user data.
3. **Confirmation Page (confirm.jsp)**: Displays submitted user data using `<jsp:getProperty>`.
4. **Include Header and Footer (header.jsp, footer.jsp)**: Common header and footer included on multiple pages using `<jsp:include>`.
5. **Error Page (error.jsp)**: Redirects to this page if there's an issue with user data, using `<jsp:forward>`.

Project Structure

```

ActionTagDemo/
├── src/
│   └── User.java           // JavaBean to hold user data
├── WebContent/
│   ├── index.jsp          // Home page with a form
│   ├── confirm.jsp        // Displays user data
│   ├── header.jsp         // Header to include
│   ├── footer.jsp         // Footer to include
│   └── error.jsp          // Error page for redirection
└── WEB-INF/
    └── web.xml             // Deployment descriptor
  
```

Step 1: Create a JavaBean (User.java)

In the `src` folder, create a `User` class that will hold user data:

```

java

package com.example;

public class User {
    private String name;
    private String email;

    public User() {
        // No-argument constructor
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }
}
  
```

Step 2: Create JSP Pages

index.jsp - Home Page with Form

This page contains a form where the user can enter their name and email.

```
jsp

<jsp:include page="header.jsp" />

<!DOCTYPE html>
<html>
<head>
    <title>User Information</title>
</head>
<body>
    <h2>Enter User Information</h2>
    <form action="confirm.jsp" method="post">
        Name: <input type="text" name="name"/><br/>
        Email: <input type="text" name="email"/><br/>
        <input type="submit" value="Submit"/>
    </form>
</body>

<jsp:include page="footer.jsp" />
</html>
```

confirm.jsp - Confirmation Page to Display User Data

In this page, we'll use `<jsp:useBean>`, `<jsp:setProperty>`, and `<jsp:getProperty>` to handle and display user data.

```
jsp

<jsp:include page="header.jsp" />

<jsp:useBean id="user" class="com.example.User" scope="request"/>
<jsp:setProperty name="user" property="name" param="name"/>
<jsp:setProperty name="user" property="email" param="email"/>

<%
    if (user.getName() == null || user.getEmail() == null) {
        response.sendRedirect("error.jsp");
    }
%>

<!DOCTYPE html>
<html>
<head>
    <title>Confirmation</title>
</head>
<body>
    <h2>Confirmation Page</h2>
    <p>Name: <jsp:getProperty name="user" property="name"/></p>
    <p>Email: <jsp:getProperty name="user" property="email"/></p>
</body>

<jsp:include page="footer.jsp" />
</html>
```

header.jsp - Header File to Include

This file contains a simple header that will be included in all pages using `<jsp:include>`.

```
jsp

<!DOCTYPE html>
<html>
<head>
    <title>Header</title>
</head>
<body>
    <header>
        <h1>Welcome to the User Information Application</h1>
        <hr/>
    </header>
</body>
</html>
```

footer.jsp - Footer File to Include

This file contains a simple footer that will be included in all pages using `<jsp:include>`.

```
jsp

<!DOCTYPE html>
<html>
<head>
    <title>Footer</title>
</head>
<body>
    <footer>
        <hr/>
        <p>&copy; 2024 My Web Application</p>
    </footer>
</body>
</html>
```

error.jsp - Error Page

This page will be used for handling cases where user data is missing or invalid, and it will be accessed using `<jsp:forward>` or `response.sendRedirect`.

```
jsp

<jsp:include page="header.jsp" />

<!DOCTYPE html>
<html>
<head>
    <title>Error</title>
</head>
<body>
    <h2>Error</h2>
    <p>There was an issue with the data submitted. Please go back and try again.</p>
    <a href="index.jsp">Back to Home</a>
</body>

<jsp:include page="footer.jsp" />
</html>
```

Step 3: Configure web.xml

In the `WEB-INF` folder, create or update `web.xml` as follows to set up the welcome page and configuration.

```
xml

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
        http://xmlns.jcp.org/xml/ns/javaee/web-app_3_1.xsd"
    version="3.1">

    <display-name>ActionTagDemo</display-name>

    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
</web-app>
```

Step 4: Deploy and Run the Application

1. Right-click the project in Eclipse and select **Run As > Run on Server**.
2. Choose the server (like Tomcat) and click **Finish**.
3. Open your browser and go to `http://localhost:8080/ActionTagDemo` to see the main page.

Explanation of JSP Action Tags Used

- `<jsp:useBean>` : Instantiates the `User` bean or retrieves it from the specified scope if it already exists.
- `<jsp:setProperty>` : Sets the properties of the `User` bean based on form data from `index.jsp`.
- `<jsp:getProperty>` : Retrieves and displays the `User` bean properties on the confirmation page.
- `<jsp:include>` : Includes the header and footer files dynamically in multiple pages.
- `<jsp:forward>` (or `response.sendRedirect`) : Redirects to `error.jsp` if the user data is invalid or missing.



ANNA UNIVERSITY

JSP Expression Language (EL) Tags

EL simplifies the syntax for accessing objects and properties, making JSP pages more readable.

Example of EL for Displaying a Value

```
jsp

<p>User Name: ${user.name}</p>
```

Example with JSTL Core Tags

Using the `c` prefix, these tags allow conditional logic and iteration within JSP.

1. `<c:if>`

Conditionally displays content.

```
jsp

<c:if test="${user.age >= 18}">
  <p>Welcome, adult user!</p>
</c:if>
```

2. `<c:choose>`, `<c:when>`, `<c:otherwise>`

Similar to an `if-else` statement.

```
jsp

<c:choose>
  <c:when test="${user.gender == 'male'}">
    <p>Hello, Mr. ${user.name}</p>
  </c:when>
  <c:otherwise>
    <p>Hello, Ms. ${user.name}</p>
  </c:otherwise>
</c:choose>
```

3. `<c:forEach>`

Loops through a collection of items.

```
jsp

<c:forEach var="item" items="${productList}">
  <p>Product Name: ${item.name}</p>
</c:forEach>
```

4. `<c:set>`

Sets a variable in a specific scope.

jsp

```
<c:set var="discount" value="10" scope="session" />
```

Example: Combining JSP Tags

The following is an example of a JSP page that uses a combination of scripting, directives, actions, and JSTL tags.

```

1 <%@ page language="java" contentType="text/html; charset=UTF-8"%>
2 <%@ page import="java.util.ArrayList"%>
3 <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
4
5 <%
6     ArrayList<String> productList = new ArrayList<>();
7     productList.add("Laptop");
8     productList.add("Smartphone");
9     productList.add("Tablet");
10    request.setAttribute("productList", productList);
11 %>
12
13 <!DOCTYPE html>
14 <html>
15 <head>
16 <title>Product List</title>
17 </head>
18 <body>
19     <h1>Available Products</h1>
20
21     <c:forEach var="product" items="${productList}">
22         <p>Product: ${product}</p>
23     </c:forEach>
24
25     <jsp:include page="footer.jsp" />
26 </body>
27 </html>

```